ESSENTIALS OF ARTIFICIAL INTELLIGENCE
INDIVIDUAL CONTRIBUTION SHEET

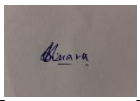
Student Name	TTC Jayasinghe	**Registration No.**	D/DBA/25/0016
Degree / Intake	DBA / 42	**Group No.**	40
Project Title	AI-Powered Dynamic Public Transport Fleet Optimizer (100)		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Dataset development (ETL)	Built the ETL that converts the Kaggle public transport parquet into the Route 100 dataset: ~800k rows stratified by (direction, hour), stop mapping via stop_id % 8.	Python, pandas, NumPy, PyArrow	Evidence 1: stratified_sample() code, backend/process_hybrid_dataset.py
Colombo demand calibration	Developed the hourly, direction, stop-weight and calendar multiplier tables and the global scale so rush-hour loads reflect Galle Road demand.	Rule-based demand modelling (NumPy / pandas)	Evidence 2: STOP_WEIGHTS and hourly/direction multiplier code (AM/PM rush multipliers; Panadura to Pettah stop weights) Evidence 5: Slides 3 & 4, Route100_Fleet_Optimizer_Group40.pptx
Feature engineering	Derived calendar and time features: day of week, weekend, Sri Lanka public holidays, school holiday, payday week, AM/PM peak, school rush and month.	pandas, holidays (LK) library	Evidence 4: holidays.country_holidays('LK') integration code Evidence 3: Dataset summary table
Event and rain scenarios	Developed synthetic event and rainfall indicators using predefined probabilities and demand multipliers. A fixed random seed was used to support reproducibility.	NumPy random generator (seed 42), pandas	Evidence 3: Dataset summary table
Data / model evaluation support	Checked dataset statistics (event, rain, holiday and school-rush rates, weekday-peak p90) and supported analysis of the per-group model metrics.	pandas, metrics review	Evidence 5: Slides 3 & 4 (commute realism context)

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  Date: 17/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

Evidence

Evidence 1: stratified_sample()

```
 def stratified_sample(df: pd.DataFrame, n: int, seed: int = SEED) ->
pd.DataFrame:
    """Sample ~n rows proportional to (trip_direction, hour) strata."""
    work = df.copy()
    work["_hour"] = ((work["trip_departure"] // 3600) % 24).astype(int)
    work["_stratum"] = (
        work["trip_direction"].astype(str) + "_" + work["_hour"].astype(str)
    )

    counts = work["_stratum"].value_counts()
    alloc = (counts / counts.sum() * n).round().astype(int)
    # Fix rounding so total == n
    diff = n - int(alloc.sum())
    if diff != 0:
        order = alloc.sort_values(ascending=False).index
        for i in range(abs(diff)):
            key = order[i % len(order)]
            alloc[key] = max(0, alloc[key] + (1 if diff > 0 else -1))

    rng = np.random.default_rng(seed)
    parts: list[pd.DataFrame] = []
    for stratum, k in alloc.items():
        if k <= 0:
            continue
        group = work[work["_stratum"] == stratum]
        take = min(k, len(group))
        idx = rng.choice(group.index.to_numpy(), size=take, replace=False)
        parts.append(work.loc[idx])

    sampled = pd.concat(parts, axis=0)
    if len(sampled) > n:
        sampled = sampled.sample(n=n, random_state=seed)
    elif len(sampled) < n:
        remaining = work.drop(index=sampled.index, errors="ignore")
        need = n - len(sampled)
        if len(remaining) > 0:
            extra = remaining.sample(n=min(need, len(remaining)),
random_state=seed)
            sampled = pd.concat([sampled, extra], axis=0)

    return sampled.drop(columns=["_hour", "_stratum"]).reset_index(drop=True)
```

Evidence 2: Colombo demand multipliers and STOP_WEIGHTS

```
STOP_WEIGHTS = {
    "Panadura": 0.85,
    "Moratuwa": 0.90,
    "Mt_Lavinia": 0.95,
    "Dehiwala": 1.00,
    "Wellawatta": 1.05,
    "Bambalapitiya": 1.15,
    "Kollupitiya": 1.25,
    "Pettah": 1.30,
}
# Scale base passengers so weekday AM/PM peak corridor demand lands in 800–1200
pax,
# yielding realistic peak headways of 4–5 minutes (buses=12–15) and midday
headways of 8–12 minutes.
GLOBAL_SCALE = 18.0
# Term-weekday school start (07) / finish (13–14) demand bump.
SCHOOL_RUSH_HOURS = (7, 13, 14)
SCHOOL_RUSH_SCALE = 1.35
```

```
HOURLY_MULTIPLIERS_WEEKDAY = {
    0: 0.10, 1: 0.08, 2: 0.08, 3: 0.08, 4: 0.08,
    5: 0.09,   # Very very less crowd (~20-25 pax, empty)
    6: 0.15,   # Less crowd (~50-60 pax)
    7: 1.15,   # Crowded (AM rush starts)
    8: 1.65,   # Crowded (AM peak)
    9: 0.55,   # Little crowded (post-rush taper)
    10: 0.12,  # Not much / very low baseline empty (~40 pax)
    11: 0.11,  # Same as 10 (~40 pax)
    12: 0.65,  # Crowded (lunch / dismissals)
    13: 0.75,  # Crowded (primary school close)
    14: 1.45,  # Very crowded (secondary schools close/tuition)
    15: 0.65,  # Crowded (after-school transition)
    16: 1.05,  # Crowded (outbound office departures)
    17: 2.10,  # Very very crowded (peak office rush)
    18: 2.30,  # Also same: very very crowded peak rush
    19: 0.70,  # Crowded (evening taper)
    20: 0.15,  # Less crowd (shops close, buses empty out)
    21: 0.09,  # Very less crowded (night service empty)
    22: 0.08,  # Very less crowded (empty)
    23: 0.08,  # Very less crowded (empty)
}
```

```
def direction_multiplier(direction: str, hour: int) -> float:
    inbound = direction == "Inbound_to_Pettah"
    if hour in (7, 8):
        return 1.35 if inbound else 0.85
    if hour in (13, 14):
        return 0.90 if inbound else 1.20
    if hour in (16, 17, 18):
```

```
        return 0.80 if inbound else 1.40
    if hour == 19:
        return 0.85 if inbound else 1.20
    return 1.0

def apply_colombo_demand(df: pd.DataFrame) -> pd.Series:
    """Peak × direction × stop × calendar × school rush × global scale (before
event/rain)."""
    soft = (df["is_weekend"] == 1) | (df["is_public_holiday"] == 1)
    peak = [
        peak_multiplier(int(h), bool(s))
        for h, s in zip(df["hour"].to_numpy(), soft.to_numpy())
    ]
    direction = [
        direction_multiplier(d, int(h))
        for d, h in zip(df["direction"].to_numpy(), df["hour"].to_numpy())
    ]
    stop = df["stop_name"].map(STOP_WEIGHTS).fillna(1.0).to_numpy()
    cal = df.apply(calendar_multiplier, axis=1).to_numpy()
    rush = np.where(df["is_school_rush"].to_numpy() == 1, SCHOOL_RUSH_SCALE, 1.0)
    base = df["passenger_count"].to_numpy(dtype=np.float64)
    scaled = (
        base
        * np.array(peak)
        * np.array(direction)
        * stop
        * cal
        * rush
        * GLOBAL_SCALE
    )
    return np.maximum(0, np.round(scaled)).astype(int)
```

Evidence 3: Dataset summary table

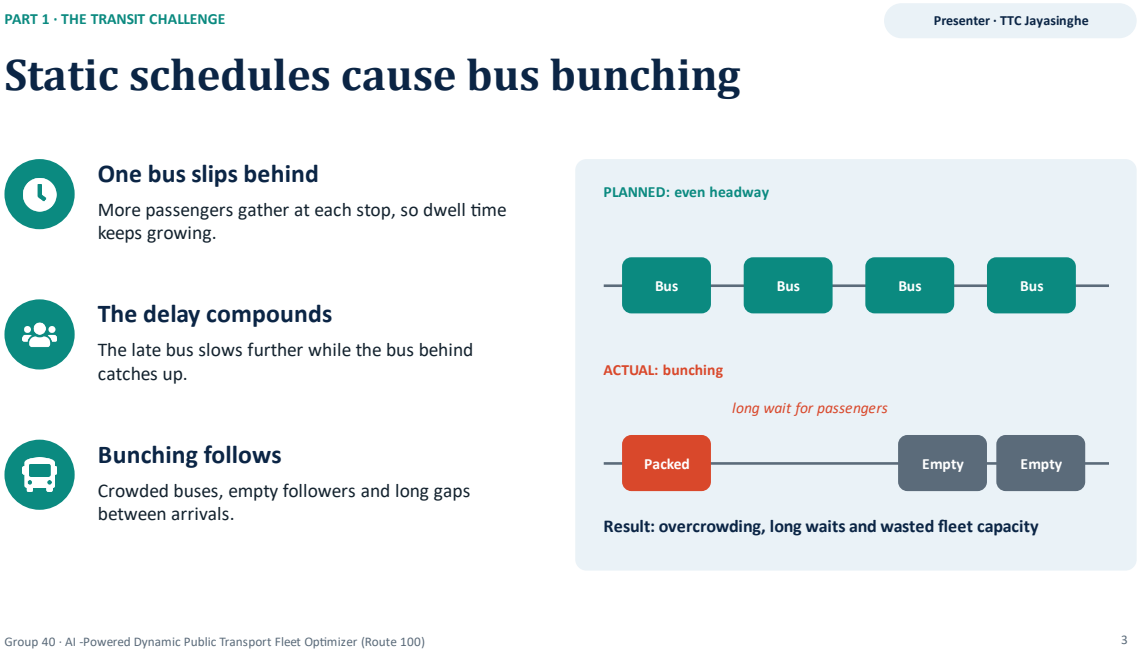
Column Name	Data Type	Description / Logic
`date`	String (`YYYY-MM-DD`)	Calendar operating date
`hour`	Integer (`0–23`)	Hour of bus departure
`direction`	Categorical	`Inbound_to_Pettah` (morning rush) or `Outbound_to_Panadura` (evening rush)
`stop_name`	Categorical	8 Galle Road stops (`Panadura` $\to$ `Pettah`)
`is_am_peak`	Binary (`0/1`)	1 if hour is 07:00, 08:00, or 09:00
`is_pm_peak`	Binary (`0/1`)	1 if hour is 16:00, 17:00, 18:00, or 19:00
`is_school_rush`	Binary (`0/1`)	1 if school transit rush (06–07 & 13–14 on term weekdays)
`is_weekend`	Binary (`0/1`)	1 for Saturday and Sunday
`is_public_holiday`	Binary (`0/1`)	1 for Sri Lankan Public/Bank/Poya holidays (via `holidays` LK library)
`is_payday_week`	Binary (`0/1`)	1 if day of month is ≥ 25 or ≤ 5 (commute surge around paydays)
`weather`	Categorical	`Clear` or `Rain` (calibrated monsoon rain probability)
`event_override`	Binary (`0/1`)	1 for special events/festivals causing high surges (~5% rows)
`passenger_count`	**Integer (Target)**	**Ground truth passenger demand (base pax $\times$ Colombo peak multipliers)**

Evidence 4: Sri Lanka Holiday Integration

```
years = sorted({int(y) for y in df["date"].dt.year.unique()})
  lk_holidays = holidays.country_holidays("LK", years=years)
  df["is_public_holiday"] = df["date"].dt.date.map(
      lambda d: 1 if d in lk_holidays else 0
  )

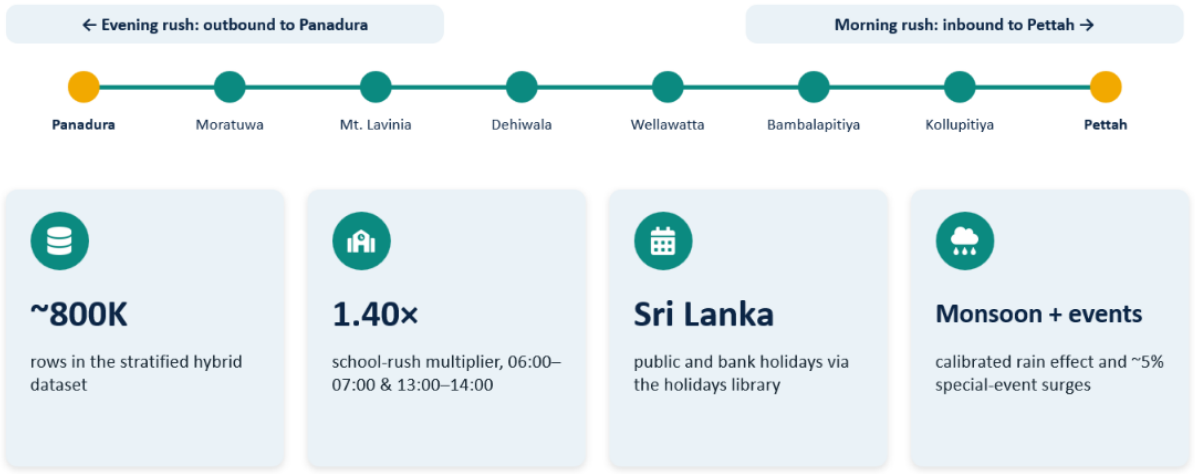
  df["is_school_holiday"] = df["date"].dt.month.isin([4, 8, 12]).astype(int)
  df["is_payday_week"] = (
      (df["date"].dt.day >= 25) | (df["date"].dt.day <= 5)
  ).astype(int)
```

Evidence 5



Slide 3

Modeling Galle Road commute realism



Slide 4